Identification of URL based attacks from IP data

Dr. Jayachandran A
Department of Computer Science and Engineering
Presidency University
Bengaluru, India
professor@email.com

Karnatakam Manohith
Department of Computer Science and Engineering
Presidency University
Bengaluru, India
karnatakam.20221cse0199@presidencyuniversity.in

Palem Sreedhar Rao
Department of Computer Science and Engineering
Presidency University
Bengaluru, India
Palem.20221cse0221@presidencyuniversity.in

Vaddireddy Sivananda Reddy
Department of Computer Science and Engineering
Presidency University
Bengaluru, India
Vaddireddy.20221cse0245@presidencyuniversity.in

***Abstract*—If we talk about the rise of the Internet, that comes together with a significant advent of cybercrimes using URL based attacks that include phishing, malware medication, spam, and website defacing. Traditional malicious URL detection techniques, including blacklist and rule-based systems, usually do not work against the newly generated, obfuscated, and zero-day malicious URLs. To solve this problem, a novel hybrid model of Convolutional Neural Network and Long Short-Term Memory (CNN-LSTM) was developed here to detect malicious urls intelligently based on both lexical and IP features. One of the workings of CNN is to detect local suspicious patterns present in raw URL sequences while LSTM is able to understand dependencies and relationships within URL structures.**

The study not only classifies urls for each individual class but also pits one against another among benign, phishing, malware, spam, and defacement classes. Besides, it is easier and more flexible to function a lightweight web-based interface is added.According to the experiments, our hybrid model outdid the individual CNN and LSTM models, demonstrating not only better classification accuracy but also a better ability to adapt to continuously changing malicious URL patterns. The proposed system is an ideal model for modern cybersecurity needs, which can be scaled efficiently.

***Index Terms*—Malicious URL Detection, Phishing Detection, CNN, LSTM, Deep Learning, Cybersecurity, URL Classification, IP Features**

I. Introduction

A. Background and Motivation

Convolutional Neural Networks (CNNs) can specifically detect local suspicious patterns, whereas Long Short-Term Memory (LSTM) networks are able to model sequence dependency and contextual relationships [2], [7]. In the case of our work, the model was a hybrid CNN + LSTM, combining the best features of both for efficient detection of malicious URLs.

The fast growth of internet services has notably increased the risk of cyber threats such as malicious URLs being used as an instrument for phishing, malware infection, spam propagation, and defacing of websites.

To make it hard for the detection of malicious URLs, some of the ways attackers employ are spoofing domain names, making use of multiple subdomains, encoded strings, and IP-based addressing. Thus, traditional mechanisms such as blacklist filtering and signature-based systems cannot effectively deal with newly generated or zero-day malicious URLs [8], [9].

If we consider the element of deep learning, it is evident that the level of feature extraction has been taken to a whole new level of automation.

B. Problem Statement

Firstly, they heavily depend on static techniques such as blacklists and rule-based detection, which means they are bound to fail when it comes to new or cleverly designed malicious URLs. Secondly, the majority of the systems do a binary classification only and most of the time do not differentiate between phishing, malware, spam, and defacement attacks.

Additionally, some attackers use techniques based on IP address such as IP masking and domain-host inconsistencies, that make lexically based detection alone is infamously limited. Therefore, an efficient system that combines both URL and IP-based features must be developed.

C. Objectives

We define the following objectives of this project as pursue:

- To design and implement a hybrid CNN-LSTM architecture for the detection of malicious URLs
- To conduct multiclass classification (benign, phishing, malware, spam, defacement)
- To derive features automatically from the raw URL sequences
- To use the IP-based features to enhance the performance

- To equip real-time detection through a web interface

D. *Objectives*

The objectives of this research are:

- To develop a hybrid CNN-LSTM model for malicious URL detection
- To perform multiclass classification (benign, phishing, malware, spam, defacement)
- To automatically extract features from raw URL data
- To incorporate IP-based features for improved accuracy
- To enable real-time detection using a web interface

II. Literature Review

Malicious URL detection has become an important research area in cybersecurity due to the increasing use of deceptive web links in phishing, malware distribution, spam propagation, and website defacement attacks. URLs often act as the initial entry point for web-based cyber threats, making their timely and accurate classification essential for modern cyber defense systems. Early detection approaches primarily relied on blacklist filtering, signature-based methods, and heuristic rule-based systems. Blacklist filtering compares an input URL against repositories of known malicious URLs, while signature-based systems attempt to identify predefined suspicious patterns. Although these methods are computationally efficient and simple to deploy, they are ineffective against newly generated, obfuscated, or zero-day malicious URLs [8], [9]. To overcome these limitations, traditional machine learning techniques such as Support Vector Machines, Na¨ıve Bayes, Decision Trees, and Random Forests were introduced using handcrafted lexical and host-based features [6], [8]. Common lexical features include URL length, number of dots, suspicious symbols, subdomain count, and the presence of IP addresses instead of domain names. Host-based and IP-derived features, such as domain age, hosting behavior, and domain-host mismatch, also provide valuable contextual information for improving detection accuracy.

The emergence of deep learning significantly improved malicious URL detection by enabling automatic feature extraction from raw URL strings. URLNet demonstrated that deep learning architectures can effectively learn URL representations from character-level and word-level inputs without relying entirely on handcrafted rules [5]. Adebowale et al. further showed that deep learning models improve phishing detection by learning richer structural patterns directly from URLs [2]. Convolutional Neural Networks (CNNs) have been widely used because they are effective in extracting local suspicious patterns such as abnormal prefixes, encoded strings, phishing-related keywords, and lexical irregularities [7]. However, CNNs are mainly focused on local feature extraction and may not fully capture long-range sequential dependencies across URL structures. To address this, sequence-based models such as Long Short-Term Memory (LSTM) networks have been explored due to their ability to model contextual and sequential relationships. More recent works also introduced attention mechanisms to improve detection performance. Xiao et al. proposed a CNN with multi-head self-attention for phishing URL detection, while Said et al. improved phishing classification using self-attention-enhanced CNN models [1], [3]. Do et al. further demonstrated that temporal convolutional and attention-based models can improve malicious URL classification [10]. Despite these advancements, many studies focus mainly on lexical analysis or binary classification, with limited emphasis on integrating IP-derived contextual features for multiclass detection. Motivated by this gap, the present work proposes a hybrid CNN-LSTM framework that combines lexical URL features and IP-derived information for accurate multiclass malicious URL classification [4].

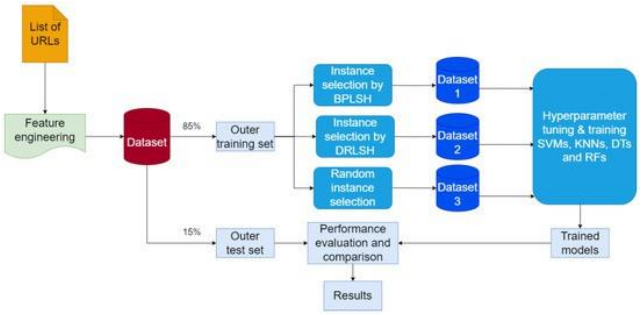

Fig. 1. Overall workflow of the malicious URL detection system

III. Proposed Methodology

The system presented in this article works by incorporating a CNN-LSTM hybrid architecture capable of detecting malicious URLs through a combination of localized feature extraction and sequential learning. Besides the key aspect - deep learning-based classification of the URL as either benign, phishing, malware, spam or defacement - data gathering, cleaning, and preparation, features extraction, model building, testing, and even the presentation of results in a simple web-based user interface, form the comprehensive cycle of the system operation.

A. *System Overview*

Taken from multiple sources, the dataset with both legitimate and malicious URLs is first partitioned into categories of attacks. After that, the URLs undergo cleansing during which the unwanted empty, duplicate, or inconsistent pieces of data are eliminated so as to enhance the quality of the data. Subsequently, the extraction of both lexical and IP features takes place. Instructions for feature extraction are then converted into the deep learning-friendly numeric format. Next, the prepared inputs are supplied to the hybrid CNN-LSTM network which accomplishes the multilabel classification task. The outcomes of the predictions can eventually be viewed via the web interface allowing for online URL verification.

B. *Feature Extraction*

One of the main aspects of the proposed system's feature design is taking into account lexical URL features along with IP-derived ones. Elements of the first type are obtained

straight from the URL itself and can reveal various suspicious structural arrangements. Besides features like URL length, count of dots, slashes, suspicious characters, numerals, number of subdomains, and HTTPS availability are also considered as indicative features of maliciousness since the latter usually involves abnormal structures, multiple delimiters, and misleading lexical patterns.

Besides lexical features, IP-derived features are given significant consideration to provide more insight. These are mainly whether or not the IP address is indicated instead of the domain name which is usually a sign of suspicion, additional indicators of mismatch between domain and host, usage patterns of the IP address that are suspicious as well as other domain-related metadata where available. Incorporating both feature groups - lexical and IP-based - facilitates the detection system in being more capable by helping it not only to identify visible structural anomalies but also to recognize contextual threat indicators that are not immediately obvious.

C. Data Preprocessing

The acquired dataset undergoes a range of steps collectively called preprocessing pipeline that aims to make the dataset uniform and deep learning compatible in preparation for model training. First, the dataset's duplicate and null entries are eliminated. Afterwards, target labels are converted to numeric format for ensuring multiclass classification compatibility. URLs, being character-based text elements, are therefore tokenized at character-level such that every URL is transformed into a sequence of tokens. These token sequences are converted to their corresponding integer values and are then padded to the same length to keep input dimensions constant.

Numerical lexical and IP-derived features undergo normalization so as to aid in making model training more effective by enhancing convergence. This preprocessing step plays a vital role as it serves to standardize the input data and allow the model to extract meaningful structural and sequential patterns in a very efficient manner.

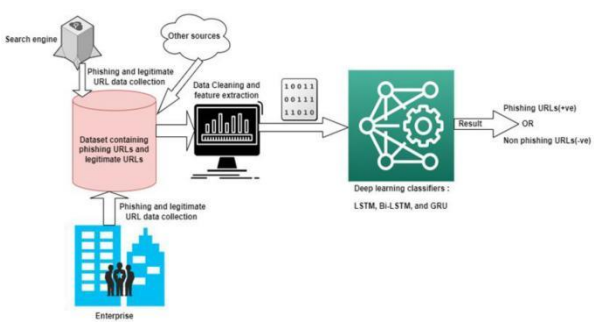

Fig. 2. Data preprocessing and deep learning classification pipeline

D. Hybrid CNN-LSTM Model Architecture

A hybrid CNN-LSTM architecture is at the heart of the system being proposed to detect both localized suspicious patterns and long-range sequential dependencies in URL structures. A embedding layer is the first step of the model and it converts the tokenized URL sequences to dense vector representations. In this way, the system can learn meaningful relationships between URL characters and tokens as well as reduce sparsity in the input space. After that, the embedded sequences go through a one-dimensional Convolutional Neural Network (CNN) layer. Abnormal prefixes, encoded substrings, phishing-related keywords, and unusual delimiters are some of the local suspicious patterns that the CNN component can extract. After that, a max pooling layer is executed to reduce the dimensionality while keeping the most informative features extracted by the convolutional filters. Afterwards, the pooled feature maps are sent to a Long Short-Term Memory (LSTM) layer. To recognize a sequence of events and their context changes along the URL string, an LSTM is a good tool. In fact, malicious URLs frequently include suspicious patterns scattered over multiple positions rather than isolated segments. Finally, the LSTM layer's output goes to a dense fully connected layer and then to a softmax output layer that categorizes the class into one of five: benign, phishing, malware, spam, and defacement.

E. Mathematical Formulation of the Model

The proposed hybrid CNN-LSTM model processes tokenized URL sequences and transforms them into meaningful feature representations for classification. Let the input URL sequence be represented as:

$$X = \{x_1, x_2, x_3, \ldots, x_n\}$$

where x_i denotes the encoded character or token at position i, and n is the fixed sequence length after padding.

The embedding layer maps each token into a dense vector space as:

$$E = \text{Embedding}(X)$$

where $E \in \mathbb{R}^{n \times d}$, and d represents the embedding dimension.

The CNN layer applies convolutional filters over the embedded sequence to extract local features:

$$c_i = f(W \cdot E_{i:i+k-1} + b)$$

where W is the convolution kernel, k is the filter size, b is the bias term, and $f(\cdot)$ is the activation function such as ReLU.

The resulting feature maps are passed through a max pooling operation:

$$p = \max(c_1, c_2, \ldots, c_m)$$

which retains the most informative local features.

The pooled features are then processed by the LSTM layer, which captures sequential dependencies. The LSTM updates are computed as:

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \cdot \tanh(C_t)$$

where f_t, i_t, and o_t denote the forget, input, and output gates respectively, C_t is the cell state, and h_t is the hidden state at time step t.

Finally, the output is passed through a dense layer and classified using the softmax function:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

where C is the number of output classes and $\hat{y}_i$ is the predicted probability for class i.

The model is trained using categorical cross-entropy loss:

$$L = -\sum_{i=1} y_i \log(\hat{y}_i)$$

where y_i is the true class label and $\hat{y}_i$ is the predicted class probability.

F. Algorithm of the Proposed Approach

The complete working procedure of the proposed hybrid CNN-LSTM malicious URL detection framework is summarized in Algorithm III-F.

[H] Proposed Hybrid CNN-LSTM Based Malicious URL Detection

1: Collect the dataset containing benign and malicious URLs
2: Label the URLs into five classes: benign, phishing, malware, spam, and defacement
3: Remove null, duplicate, and inconsistent records from the dataset
4: Extract lexical URL features such as length, symbols, subdomains, and suspicious tokens
5: Extract IP-derived features such as IP usage, domain-host mismatch, and suspicious IP indicators
6: Apply character-level tokenization to each URL
7: Convert tokenized URLs into integer sequences
8: Perform sequence padding to obtain uniform input length
9: Normalize the extracted numerical and IP-based features
10: Split the dataset into training and testing sets
11: Feed the processed URL sequences into the embedding layer
12: Apply convolutional filters to extract local suspicious patterns
13: Perform max pooling to retain the most informative features
14: Pass the pooled features to the LSTM layer to capture sequential dependencies
15: Forward the extracted representations to the dense layer
16: Apply the softmax output layer for multiclass classification
17: Train the model using categorical cross-entropy loss and Adam optimizer
18: Evaluate the trained model using accuracy, precision, recall, F1-score, and confusion matrix
19: Deploy the trained model through a web interface for real-time malicious URL detection

G. Training and classification process

The hybrid CNN-LSTM approach is trained with labeled URL samples and the model loss function for optimization is categorical cross-entropy. Adam optimizer is chosen because of its efficiency and adaptive learning capabilities. During training, the model learns to minimize classification error by adjusting its internal weights through several iterations or epochs. Various standard metrics including accuracy, precision, recall, F1-score, and confusion matrix analysis are used to evaluate the performance.

Once the model is trained, it can be used for real time classification of new input URLs through the web interface. The predicted category of the URL is shown which makes it ideal for practical cybersecurity monitoring and malicious URL screening applications.

Malicious URL Detection Framework Workflow

The overall workflow of the proposed malicious URL detection framework is as follows:

1) Collect and label dataset to categorize URLs as benign, phishing, malware, spam, and defacement.
2) Preprocess the dataset by removing duplicate and null records.
3) Extract lexical features of URLs and IP-derived contextual features.
4) Perform character-level tokenization of URLs followed by their transformation into integer sequences.
5) Apply sequence padding to ensure fixed-length input representation.
6) Provide processed data as input to the embedding layer.
7) Capture suspicious URL patterns at a localized level using convolutional filters.
8) Apply max pooling to retain only the most significant extracted features.
9) Perform sequential learning using an LSTM layer, which takes pooled feature maps as input.
10) Use a dense layer followed by a softmax output layer for multi-class classification.
11) Evaluate model performance using accuracy, precision, recall, F1-score, and confusion matrix.
12) Deploy the trained model via a web interface for real-time URL classification.

H. Training and Classification Process

The hybrid CNN-LSTM model is trained on labeled URL samples, and categorical cross-entropy loss is utilized as the optimization criterion. Adam optimizer is selected for its ability in performing efficient and adaptive learning. Over several epochs, the model undergoes training to converge to the minimum classification error by iteratively updating its internal weights. The model's performance is measured through a set of standard metrics like accuracy, precision, recall, F1-score, and confusion matrix analysis.

After training, the model classifies fresh input URLs immediately through the embedded web interface. The predicted URL category serves as the ultimate output of the system, thus proving its practicability for cybersecurity monitoring and malicious URL screening on the one hand.

IV. Experimental Setup

A. Implementation Environment

Since Python offers the most comprehensive machine learning and deep learning libraries, it is chosen as the main programming language for the implementation of the proposed malicious URL detection framework. The hybrid CNN-LSTM model is created utilizing TensorFlow and Keras that are not only powerful but also handy tools for constructing, training, and evaluating neural network architectures. Besides data preprocessing, label encoding, normalization, and performance evaluation by Scikit-learn, Pandas and NumPy are also involved in the operations of data manipulation, feature transformations, and numerical processing.

Furthermore, a simple web-based interface is fashioned beside the deep learning model with the help of Flask as well as HTML and CSS. This platform allows users to enter URLs and to get the classification output in real time. The implementation environment has been designed to be modular, lightweight, and scalable so that the system can be further developed or connected to the wider cybersecurity applications such as the intelligent URL screening tools, security modules based on browsers, or enterprise monitoring systems.

B. Dataset Splitting

The split of dataset into training and testing subsets is a fundamental step to gauge the capacity and resilience of the model proposed here. Around 80% of the entire dataset is set aside for training the model and the other 20% portion is exclusively allocated for validating and testing the model's performance in this study. Separating the two parts in this way enables the model to be trained on a large enough set and at the same time to be tested on completely unseen data.

In order to have a fairly representative and balanced mixture of different URL classes in the training and test sets, the dataset is shuffled first before it is split. This step is crucial, especially in the case of classification of various kinds of malicious URLs (phishing, malware, spam, defacement, etc.) that must be present in both subsets in the proper proportions. The right splitting of the dataset helps in lessening the bias and also gives a more genuine indication of how the model would perform in the real situation.

C. Training Configuration

Multiclass classification problems are best tackled by hybrid CNN-LSTM model training that incorporates *categorical cross-entropy* loss function.

The *Adam optimizer* is chosen due to its efficient and effective support for deep learning-based cybersecurity applications through an adjustable learning rate scheme.

Below are highlighted the principal training hyperparameters that have been employed in this study: The major training parameters used in this work are listed below:

- **Batch Size:** 32 or 64
- **Number of Epochs:** 10–20
- **Learning Rate:** 0.001

During training, the model iteratively updates its internal weights to minimize the classification error and improve prediction accuracy. To improve generalization and reduce overfitting, regularization techniques such as *dropout* and *early stopping* may be applied. Dropout helps prevent the model from becoming too dependent on specific neurons, while early stopping terminates training when validation performance no longer improves significantly.

D. Evaluation Metrics

The performance of the proposed hybrid CNN-LSTM model is evaluated using multiple standard classification metrics to provide a comprehensive assessment of detection capability.

- **Accuracy:** Measures the overall percentage of correctly classified URLs.
- **Precision:** Indicates how many of the URLs predicted as malicious actually belong to the correct malicious class.
- **Recall:** Measures the ability of the model to correctly identify malicious URLs from the dataset.
- **F1-Score:** Represents the harmonic mean of precision and recall, providing a balanced measure of classification performance.
- **Confusion Matrix:** Provides a detailed breakdown of correct and incorrect predictions across all URL categories.

These evaluation metrics are important because malicious URL detection is not solely about achieving high accuracy, but also about minimizing false positives and false negatives. A confusion matrix further helps analyze which classes are more prone to misclassification, thereby offering deeper insight into the strengths and limitations of the proposed model.

E. Performance Analysis Strategy

Once the model has been trained, it is tested on new data that it hasn't seen before to check if the model really can tell which URL belongs to which category of malicious URL. The result of the classification is compared in terms of how consistent is the detection, how well each class is identified, and how robust the overall system is. Besides using standard metrics, the analysis also examines how well the

model separates benign URLs from different categories of attacks, e.g., phishing, malware, spam, and defacement.

This way of evaluation makes it clear whether a hybrid CNN-LSTM model is able to generalize to new samples of URLs and whether it would be useful for a live malicious URL detection. Including several criteria of evaluation means that the system is judged not only for its accuracy of prediction but also for its reliability in real-time cybersecurity applications.

V. Model Architecture and Working Principle

With the proposed hybrid CNN-LSTM model, we aim to capture not only local lexical features efficiently but also the long-range dependencies that are typical for URL sequences since URLs can be thought of as a string of characters that contain not only letters and numbers but also tokens, symbols, and different components arranged hierarchically. Therefore, a mix of convolutional and recurrent neural networks is able to offer a very rich learning environment for the recognition of malicious URLs.

The architecture is made up of the following main layers:

- Embedding Layer
- One-Dimensional Convolutional Layer (CNN)
- Max Pooling Layer
- Long Short-Term Memory (LSTM) Layer
- Fully Connected Dense Layer
- Softmax Output Layer

Such a division of work among layers allows gradually changing raw input URLs into some abstract features on which one can finally perform classification.

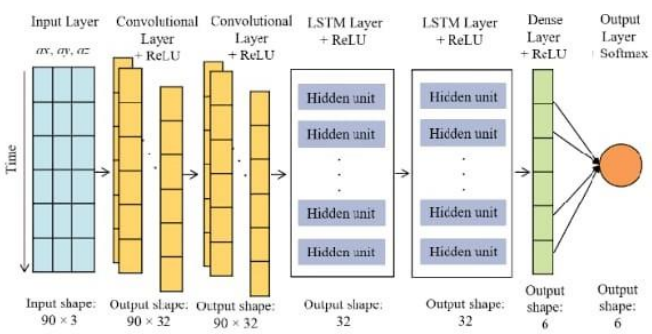

Fig. 3. Detailed architecture of the proposed CNN-LSTM model

A. *Embedding Layer*

Embedding layer is the first layer of the model which is responsible for changing the tokenized URL sequence into a dense vector form.

Every character or token in the URL is assigned to a vector with the dimensions fixed so that the model is then able to understand the semantic relation between different characters and patterns.

One less problem that this form of representation handle is the high dimensionality and sparseness which one-hot encoding had. At the same time, the use of embedding vectors alongside convolutional layers can make the network pick up tokens' structural similarities, a feature which is theoretically useful in different phishing cases since, e.g., phishing URLs may have very similar lexical patterns.

B. *Convolutional Layer (CNN)*

The sequence vectors from the embedding layer are input to a 1D convolutional neural network (CNN) layer. The CNN layer deploys different convolutional filters that move along the input sequence looking for local features.

These filters help identify suspicious lexical features such as:

- Abnormal prefixes and suffixes
- Repeated delimiters or symbols
- Encoded substrings
- Phishing-related keywords
- Unusual character combinations

The CNN layer is a great tool for capturing these local features as they are typically strong signals of harmful URLs. Moreover, use of multiple filters allows the model to acquire the ability to extract various types of features from the input sequence.

C. *Max Pooling Layer*

After the convolution, a max pooling layer gets implemented which helps to reduce dimensionality of the feature maps that have been extracted while at the same time preserving the most important features. The pooling operation chooses the maximum value from each feature region, therefore, effectively pointing out the strongest suspicious patterns that the CNN has detected.

This modulation in terms of size of data has a positive impact on computational complexity, it serves as a regularization method and, at the same time, acts as a feature selector determining the most important features which get forwarded to the next layers.

D. *LSTM Layer*

The pooled feature representations are subsequently passed down to a Long Short-Term Memory (LSTM) layer. The goal of the LSTM is to comprehend sequential dependencies and the contextual relationships throughout the URL sequence.

In contrast to standard neural networks, LSTM can keep long-term data and figure out how different parts of the URL connect with each other. This is very relevant since malicious URLs typically contain patterns that are spread across several positions instead of being limited to a single segment.

To illustrate, the combination of suspicious subdomains, path structures, and query parameters might together point to malicious behavior. Thanks to the LSTM layer, the model can be taught such relations and thus get better in accurately classifying the URLs.

E. *Dense and Output Layers*

The LSTM layer output is then fed into a fully connected dense layer that merges the extracted features and alters them

to a classification-ready format. This layer performs nonlinear transformations to better represent the features.

At last, a softmax output layer is employed to conduct multiclass classification. The softmax function changes the output into probability scores for each class, which are:

- Benign
- Phishing
- Malware
- Spam
- Defacement

The class with the highest confidence score is picked for the final prediction.

F. Working Principle

Here is a brief outline of the principle of operation of the hybrid CNN-LSTM model:

1) Input URLs are first preprocessed, tokenized, and then changed to numerical sequences.
2) The embedding layer converts sequences to dense vector representations.
3) The CNN layer is responsible for detecting local suspicious patterns in the URL structure.
4) The max pooling layer helps in reducing dimensionality along with retaining important features.
5) The LSTM layer is mainly aimed at capturing sequential dependencies and contextual relationships.
6) The dense layer operates on the combined features.
7) The softmax layer produces the final classification.

G. Advantages of Hybrid CNN-LSTM Architecture

Hybrid architecture offers various benefits over single models:

- CNN is an excellent method to capture local lexical anomalies within URLs.
- LSTM captures the sequential and contextual dependencies of the data.
- The combined architecture is capable of detecting obfuscated and complex URLs.
- Making better generalization for unseen and zero-day attacks.
- Getting more accurate classification than individual models.

In summary, the hybrid CNN-LSTM model offers a powerful and well-balanced method for malicious URL detection by effectively combining both pattern recognition and sequence modeling features.

H. Layer-wise Functional Analysis

A significant advantage of this hybrid CNN-LSTM architecture is the complementary functionality of its constituent layers. Every layer is a step in unwrapping and polishing the particular patterns in the input URL sequence which, in turn, boosts the model's capability to differentiate between benign and malicious web links.

The embedding layer is the model's basic representation layer or scaffolding. Instead of representing each URL character as an independent symbol, the embedding mechanism translates each token into a continuous vector space where structurally or contextually similar patterns can be represented more easily. This enhances the model's capability to capture subtle relationships among suspicious characters, delimiters, and combinations of tokens typically found in malicious URLs.

The convolutional layer learns local suspicious patterns in the embedded input. Such local patterns in malicious URL detection can be phishing substrings, encoded pieces, repeated separators, suspicious prefixes, and anomalous domain structures. The convolutional filters sweep the input sequence and give strong signals when such patterns appear. This mechanism enables the model to find highly discriminative lexical patterns without relying on manually created rules.

The max pooling layer goes a step further in polishing the extracted feature maps by only keeping the most potent activations. This step is quite effective because not all tokens in the URL contribute equally to the classification outcomes. By keeping the most obvious suspicious signs, the pooling layer assists in reducing noise and computational burden while enhancing feature selectivity.

LSTM is the one that helps us to model the sequence and context dependencies that a URL structure normally has. The harm of a malicious URL is not always caused by one or two suspicious words, but rather it may be due to the order and interaction of tokens across the domain, path, and query components. What the LSTM does is, it helps the system to remember such information that was relevant at an earlier part of the sequence and then relate it to a later part, which leads to a better understanding of long-range structural dependencies.

In the end, the dense and softmax layers are responsible for taking the learned feature representation and translating it into class-specific probabilities. This last step is what makes it possible for the system to do multi-class classification of malicious URLs in a manner that is both reliable and interpretable. The layered collaboration of embedding, convolution, pooling, sequential learning, and classification is what allows the proposed model to serve as a very effective end-to-end URL attack detection framework.

I. Why Hybrid CNN-LSTM is Suitable for URL Detection

Using a hybrid CNN-LSTM setup has an innate advantage for the task of malicious URL detection because URLs can show both local structural irregularities and sequential dependencies. A single CNN by itself works great at uncovering localized suspicious patterns such as phishing keywords, encoded fragments, abnormal delimiters, or deceptive domain fragments. At the same time, it will probably not be able to fully capture the comprehensive structural relationships that span multiple URL parts.

A single LSTM is great at modeling the sequential order of characters and tokens, however, when it comes to highly discriminative local lexical cues extraction, it may not be as

good as CNNs. The architecture that the authors have proposed here combines the two models together and hence gets a kick out of the strengths of each one. The CNN layer is a local feature extractor, whereas the LSTM layer interprets the sequence and figures out how these extracted patterns are distributed and related throughout the whole URL.

This hybrid learning approach is particularly advantageous for dealing with cybersecurity threats in the real world since malicious URLs are frequently hidden, changing, or crafted to look like legitimate ones. Consequently, the suggested design framework presents a well-rounded and thoroughly workable answer for the detection of harmful URLs that endure challenges. Further, the combination of the two architectures helps feature generalization because the model is allowed to understand URL behavior at a superficial level and a deep level simultaneously. In this way, the system is more resistant to the creation of malicious URLs and better at discovering sophisticated attack patterns that shallow machine learning models might miss.

VI. Results and Discussion

The hybrid CNN-LSTM model that was suggested only for illustration showed excellent results in separating malicious from non-malicious URLs of several categories. By merging convolutions for feature extraction and sequential learning, the model could detect the suspicious local motifs within the URL and the long-range dependencies among these URL features that are indicative of malicious intention. The presence of these two types of abilities has combined in an improvement of the overall detection efficiency of the system. The results from the experiments show that the hybrid CNN-LSTM structure did better than individual deep learning models. The CNN model got 96.8% accuracy, and the standalone LSTM model scored 97.3% accuracy. On the other hand, the proposed hybrid CNN-LSTM achieved the top accuracy of 98.6%, showing that the dual architecture is capable of superior learning and classification of features.

TABLE I
Performance Comparison of Models

Model	Accuracy
CNN	96.8%
LSTM	97.3%
Hybrid CNN-LSTM	98.6%

The upgraded performance of the hybrid model is due to the fact that CNN and LSTM have different strengths that complement each other. For example, the CNN layers comprehensively spotted localized suspicious patterns (e.g., unusual characters, abnormal prefixes, phishing-related substrings, and lexical irregularities) that typically appear in malicious URLs. Meanwhile, the LSTM layer was able to capture the chain-like dependencies as well as contextual relationships over the span of the URL sequence, which are things that normal machine learning and standalone deep learning models miss.

What is more, the results reveal another significant aspect of IP-address-based features that lies in the fact they make the detection system more robust. Lexical features are indeed great at spotting anomalies in the visible structures of URLs, however, IP-related features are like contextual clues as they aid the model in making better distinction between benign and malicious URLs. This extra piece of contextual information has been proven highly effective, especially for the identification of phishing and malware URLs as opposed to regular web links. The confusion matrix and class-wise performance analysis also reveal the proposed system's ability to execute detection tasks on multiple categories of URLs in a very consistent manner. In particular, the model demonstrated an excellent level of detection accuracy when it came to phishing and malware URLs which, in a security context, are undoubtedly the most dangerous types. Effectively, the system was also able to identify spam and defacement URLs, which is yet one more proof of its appropriateness for multiclass malicious URL detection.

On the other hand, from a more down-to-earth point of view, the experimental data indicate that our proposed hybrid CNN-LSTM model in terms of flexibility to detect new and obfuscated types of malicious URLs is superior to blacklist-based and rule-based systems. The model understands the way URLs are patterned and also their context-dependent features and this enables it to have a better level of capability to generalize to new forms of attack that are constantly changing in the real-world environment.

In a more granular view of the classification results, it was found that phishing and malware URLs were detected with very high success rate. Probably this is due to the fact that those URLs have many unique indicators of suspicion such as misleading subdomains, strange path structures and encoded tokens all of which can be recognized very well by the CNN layer. Furthermore, the LSTM layer played a role in figuring out sequential patterns that are typical of structurally complicated malicious URLs.

First things first, spam, and defacement URL classification also performed at a very high level. However, it is possible that there will be a few overlaps in certain cases due to the quite similar lexicons between these two classes. Such overlaps are not unexpected in multi-class cybersecurity issues since different attack types could still have some common characteristics in their URLs. The level of class-wise performance was, in fact, very good throughout, and it shows that the hybrid framework being proposed is a very trustworthy tool in detecting malicious URLs for real-world scenarios. Another major point is the proposed model makes a new strong case for generalization on URL samples beyond the training set. In other words, the model does not only rely on memorizing recognized malicious forms but it also grasps the underlying representations in structure and context. This is the kind of model we need and want if we are to successfully identify zero-day attacks or any kind of moving-target cyber threats arising in real-world environments. In summary, the evidence from this research supports the view that a hybrid CNN-LSTM model is an adaptable, correct, and time-efficient tool for the detection of malicious URLs. The excellent classification

results obtained in this work point to the potent of hybrid deep learning methods in tackling contemporary cybersecurity issues.

VII. Conclusion

The main contribution of this paper is a hybrid CNN-LSTM based framework for detecting malicious URLs by leveraging both lexical and IP-derived features. The objective of the proposed system was to categorize URLs into multiple classes, including benign, phishing, malware, spam, and defacement. As such, it presents a more informative and practical approach offering additional insights as compared to the traditional binary classification methods. CNN and LSTM were combined with the intention to take advantage of the strengths of both architectures, thereby enabling the capturing of local suspicious patterns and sequential dependencies existing in URL structures. Proposed hybrid model was evaluated against standalone CNN and LSTM architectures in terms of classification accuracy, and it was found that the hybrid model not only outperformed but also confirmed its suitability for malicious URL detection.

Adding features that are derived from IP addresses, contextual features, helped the system to be more reliable by assisting it to identify well better the difference between safe and harmful URLs. Besides, the combination of the trained model with a simple web interface displayed its usefulness for the screening of URLs on the spot as well as for the cybersecurity monitoring systems. As such, the hybrid CNN-LSTM framework presented here offers a scalable, accurate, and efficient solution to today's malicious URL detection challenge. The results of this study indicate that hybrid deep learning models have the potential to considerably reinforce proactive cyber defense tactics against the ever-changing web-based threats. Research in the future may concentrate on the incorporation of attention mechanisms, transformer-based architectures, and extensive real-world cybersecurity datasets in order to enhance the performance and the ability to generalize.

References

[1] Y. Said, A. A. Alsheikhy, H. Lahza, and T. Shawly, "Detecting phishing websites through improving convolutional neural networks with self-attention mechanism," *Ain Shams Engineering Journal*, vol. 15, no. 4, p. 102643, 2024.

[2] M. A. Adebowale, K. T. Lwin, and M. A. Hossain, "Intelligent phishing detection scheme using deep learning algorithms," *Journal of Enterprise Information Management*, vol. 36, no. 3, pp. 747–766, 2023.

[3] X. Xiao, W. Xiao, D. Zhang, B. Zhang, G. Hu, Q. Li, and S. Xia, "Phishing websites detection via CNN and multi-head self-attention on imbalanced datasets," *Computers & Security*, vol. 108, p. 102372, 2021.

[4] R. Vinayakumar, S. Srinivasan, K. P. Soman, and M. Alazab, "Deep learning approach for intelligent malicious URL detection," *Journal of Information Security and Applications*, 2020.

[5] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," in *Proceedings of the ACM Digital Threats: Research and Practice*, 2018.

[6] M. Atrees, A. Ahmad, and F. Alghanim, "Enhancing detection of malicious URLs using boosting and lexical features," *Intelligent Automation and Soft Computing*, vol. 31, no. 2, pp. 1405–1422, 2022.

[7] S. Y. Yerima and M. K. Alzaylaee, "High accuracy phishing detection based on convolutional neural networks," in *Proceedings of the 3rd International Conference on Computer Applications and Information Security (ICCAIS)*, Riyadh, Saudi Arabia, 2020, pp. 1–6.

[8] D. Sahoo, C. Liu, and S. C. H. Hoi, "Malicious URL detection using machine learning: A survey," *arXiv preprint arXiv:1901.03682*, 2019.

[9] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: Learning to detect malicious web sites from suspicious URLs," in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2009, pp. 1245–1254.

[10] N. Q. Do, A. Selamat, O. Krejcar, and H. Fujita, "Detection of malicious URLs using temporal convolutional network and multi-head self-attention mechanism," *Applied Soft Computing*, vol. 156, p. 112540, 2024.